ITA03-MOBILE COMPUTING ASSIGNMENT - 4A

ASSIGNMENT-4 – CO4

CO4 Coverage: Different Mobile OS (android/IOS/BB) - Architecture of Android, Blackberry, IOS & windows - Framework of Android - Blackberry - IOS & windows. Psion EPOC -Palm OS -WML (Wireless Markup Language)-J2ME/JME-Symbian Comparing and Contrasting architectures of Android, iOS and Windows - Underlying
OS (Darwin vs. Linux vs. Win 8) - Kernel structure and native level programming - Runtime (Objective-C vs. Dalvik vs. WinRT) - Approaches to power management – Security.

Mobile Application Development Environment – Assignment 4, Part I

SIMATS ENGINEERING · Architectural Analysis and Design Through Structured Reasoning

Course	Mobile Application Development Environment
Course Outcome	CO4 – Analyze mobile operating systems, architectures, runtimes, security, and power management techniques
Nature of Assignment	Individual
Duration	One Week
Total Marks	50
Submission Date	5/11/2026

Purpose of the Assignment

This assignment evaluates your ability to analyse and design a mobile platform solution before building anything. You are expected to demonstrate the following progression:
Interpret the Scenario → Identify Components and Constraints → Design the Architecture or Policy → Document It Precisely → Trace a Scenario → Challenge → Refine

This is not an app-development assignment. An IDE, emulator or SDK is not required. All designs must be written in structured design notation (layer tables, step sequences, state-transition tables), following

the SIMATS Engineering guidelines for technical design documentation.

Platforms in scope: Android, iOS, BlackBerry, Windows; legacy platforms (Symbian, Palm OS, Psion EPOC, J2ME/JME, WML) where a task names them.

Task 1 – Architecture Review (6 Marks)

Diagnose and Repair a Poor Architecture Description

A junior engineer submits the following design note describing a mobile operating system:

MobileOS Design Note (Draft)

- The device has hardware at the bottom and applications at the top. ● A kernel sits in the middle and runs the applications.
- Applications are compiled and executed by the runtime.
- The OS keeps applications secure using permissions.
- The OS saves battery when needed.
- Developers use the framework to build applications.

Task 1.1 – Diagnose the Description | 2 Marks

Identify at least **five** technical deficiencies in the note. Do not focus on wording or formatting. Examine aspects such as:

- Missing layers (drivers, hardware abstraction, native libraries, framework services)
- Kernel type and the exact responsibilities assigned to it
- Meaning of "runtime": what it executes, and when code is translated ● Meaning of "permissions": who grants them, when, and where they are enforced
- Trigger and mechanism of "saves battery when needed"
- Boundary and communication path between an application and the OS

For each deficiency, analyse briefly why another engineer could not reliably build or evaluate a system from the note as written.

Task 1.2 – Repair the Description | 3 Marks

Choose **one** OS (Android, iOS or Windows) and rewrite the note as a complete, precise layered architecture specification. For every layer state its name, its responsibility, what it exposes to the layer above, and what it depends on in the layer below. The kernel, the runtime, the security enforcement point and the power management component must each be located explicitly.

Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, analyse why your rewritten specification is a precise architectural description rather than a marketing overview or informal English.

Evidence Expected: A reasoned diagnosis, a corrected layered specification and a concise justification.

Task 2 – From Platform Requirement to Architectural Comparison (8 Marks) Android vs

iOS vs Windows

A start-up is building a secure messaging application that must receive messages in the background and run on Android, iOS and Windows devices. Before development begins, the team must understand how differences in kernel structure and runtime will affect the product. You are not given the comparison; you must construct it.

Task 2.1 – Model the Requirement | 2 Marks

Before comparing, identify:

- Input (what is being compared)
- Output (what the comparison must let the team decide)
- Objective
- Constraints (background execution, security, battery, responsiveness) ● Decision to be made
- The architectural information needed to judge each platform

Task 2.2 – Formulate a Hypothesis | 1 Mark

State in one precise sentence which architectural difference (for example kernel structure, runtime model or process model) you expect to affect background message delivery the most, and briefly justify why.

Task 2.3 – Build the Comparison | 3 Marks

Prepare a comparison matrix covering at least: kernel family and structure (Linux, Darwin, Windows), kernel-level programming interface, runtime environment (Dalvik, Objective-C runtime, WinRT), application framework, application isolation model, and inter-process communication. Every cell must contain a specific technical statement, not "supported" or "good". Add a final column or paragraph stating the consequence of each difference for the messaging application.

Task 2.4 – Desk-Check | 1 Mark

Trace the path of **one incoming message while the screen is off** through the architecture of two platforms of your choice. Show the important steps layer by layer: hardware or network event, kernel or driver, system service, runtime, application callback.

Task 2.5 – Challenge Your Comparison | 1 Mark

Change one condition in the scenario (for example: the application has been terminated by the OS due to memory pressure). Before tracing again:

1. Predict whether delivery behaviour will differ across the two platforms. 2. Identify the architectural element you expect to be responsible. 3. Trace only the relevant steps and state whether your prediction was correct.

Evidence Expected: Problem model, hypothesis, specific comparison matrix, layer wise trace and modified-case reasoning.

Task 3 – Runtime Environment Analysis (9 Marks)

From Source Code to Executing Process

A team wants to understand exactly what happens between a user tapping an application icon and the first screen appearing on Android (Dalvik), iOS (Objective-C runtime) and Windows (WinRT).

Task 3.1 – Define the Execution Model | 1 Mark

For each of the three platforms define precisely: what form of code is delivered to the device (bytecode, native binary, or other), who loads it, and at what point it becomes executable machine instructions (install time, load time or run time).

Task 3.2 – Develop the Launch Sequence | 2 Marks

Develop the ordered launch sequence for **one** platform, from the user action to the first rendered screen. For every step state what the step does, which component
performs it, and what must already exist for it to start. analyse the purpose of each step rather than only listing it.

Task 3.3 – Compare Runtime Strategies | 4 Marks

Produce a written analysis that covers:

- Interpretation, just-in-time compilation and ahead-of-time compilation, and which platform's runtime uses which idea
- Memory management approach of each runtime (garbage collection versus reference counting) and its effect on responsiveness and battery
- How the runtime reaches native code and kernel services (native-level programming interface)
- The effect of the runtime design on start-up time, memory footprint and portability

Conclude with a justified statement of which runtime design is most suitable for a low memory device.

Task 3.4 – Desk-Check | 1 Mark

Using your launch sequence from Task 3.2, trace a **cold start** and a **warm start** of the same application. Show which steps are executed in each case and which are skipped, and why.

Task 3.5 – analyse the Dependency | 1 Mark

analyse why certain components (kernel process and memory services, runtime initialisation) must be available before later steps (framework callbacks, application code) can run. Relate your explanation to the order in your launch sequence.

Evidence Expected: Execution model definition, justified launch sequence, runtime comparison, cold/warm trace and dependency explanation.

Task 4 – Security Forensics (7 Marks)

Find, analyse and Repair a Security Design Defect

The following is the security design of an application called *SecureNotes*:

1. At install time the app requests every available permission “for future use”. 2. The login token is saved in a file readable by other applications. 3. Updates are accepted from any source without checking who produced them. 4. All server communication uses plain, unencrypted connections.

Task 4.1 – Locate the Defects | 2 Marks

For each of the four decisions, identify the specific mobile OS security mechanism (permission model, sandboxing, code signing, secure storage, transport protection) that is bypassed or unused. analyse why the OS cannot protect the user if the design is left unchanged.

Task 4.2 – Construct an Attack Scenario | 2 Marks

Design your own realistic scenario in which a malicious application, or an attacker on the network, exploits **at least two** of these defects. Trace the attack step by step and show the exact point at which confidentiality or integrity is first lost.

Task 4.3 – Repair the Design | 2 Marks

Correct only the required decisions. Do not replace the whole application. For each repair name the OS mechanism you are relying on and state how it appears in the OS architecture of your chosen platform (Android, iOS or Windows).

Task 4.4 – Regression Check | 1 Mark

Identify one legitimate behaviour of the original application (for example, reading contacts after the user has agreed) that must still work. Demonstrate that your repairs preserve it.

Evidence Expected: Exact defects with mechanisms, student-designed attack trace, minimal repairs and regression evidence.

Task 5 – Power Management Reasoning (7 Marks)

Same Device, Different Power Demands

A fitness application keeps the processor awake permanently so that it never misses a sensor reading. Users report that the battery drains within a few hours even when the phone is untouched.

Task 5.1 – Design the Power-State Policy | 3 Marks

Design a power-state model for the device with at least three states (for example active, idle and suspended). For each state specify: what the hardware and kernel are doing, the condition that causes entry, the condition that causes exit, and which applications or services are allowed to run. Present it as a state-transition table.

Task 5.2 – Change the Assumption | 2 Marks

The application must now also receive urgent push notifications while the device is suspended.

Identify precisely which part of your previous policy becomes unsafe or insufficient. analyse how a platform mechanism (such as wake locks, scheduled background tasks or push-based wake-up) lets the device remain mostly asleep while still meeting the requirement, and what the application must stop doing.

Task 5.3 – Change the Requirement | 2 Marks

The organisation now requires a **battery-saver mode** that activates at 15% charge. Define what changes at the kernel level, the runtime level and the application level when it activates. State one user-visible trade-off and justify why it is acceptable.

Evidence Expected: State-transition design, analysis of the changed assumption, battery-saver policy and trade-off reasoning.

Task 6 – Design Challenge (13 Marks)

Platform Strategy for a Field-Data Collection System

A rural health organisation, *RuralCare*, collects patient records using three classes of device:

- **Class A:** modern Android smartphones
- **Class B:** iPhones
- **Class C:** legacy feature phones that support only J2ME/JME applications or a WML browser

Field workers are often offline for hours and must complete a ten-hour shift on one battery charge. Patient data must stay protected if a device is lost. Records collected offline must be synchronised when connectivity returns, without loss or duplication.

You are not told which platform strategy to use. You must develop the solution from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

Clearly identify the input, output, objective, constraints and decisions to be made. Do not choose a technology before formulating the requirement.

Evidence Expected: A precise abstraction of the problem.

Stage 2 – Propose the Strategy | 2 Marks

Consider possible ways of serving the three device classes, for example native applications per platform, one cross-platform codebase, a web or WML fallback for Class C, or restricting the programme to smartphones. Determine which strategy you would use and justify it from the characteristics of each class (runtime, storage, connectivity, power controls).

Important: A strategy is not justified because it is popular or convenient. analyse why its principle suits the constraints of this problem.

Evidence Expected: Reasoned strategy selection rather than only a technology name.

Stage 3 – Identify the Critical Information | 2 Marks

Build a constraint table with one row per device class and columns for: runtime environment, available security mechanisms, power management controls, offline storage options and connectivity. Then identify the information the system must retain to avoid loss or duplication during offline synchronisation, and why each item is required.

Evidence Expected: A precise constraint table and a justified list of retained information.

Stage 4 – Design the Complete Solution Specification | 4 Marks Write a complete design specification in structured notation. It must clearly show:

- Architecture for each device class (layers used, runtime, framework) ● Data capture and offline storage design
- Security design: protection of data at rest, protection in transit, behaviour if the device is lost
- Power management policy for a ten-hour shift
- Synchronisation procedure as ordered steps, including handling of an interrupted sync
- Failure handling for each device class

Another competent engineer should be able to implement the design from your specification without clarification.

Evidence Expected: A complete specification representing your own design. **Stage 5 – Desk-Check | 1 Mark**

Trace this scenario through your design: *a Class A worker is offline for six hours, the battery falls to 20%, and the phone is then lost before any synchronisation.* Show the important states, decisions and protective actions step by step, and report what data is exposed, what data is lost and what remains recoverable.

Stage 6 – Attack Your Own Design | 2 Marks

Construct **at least three** adverse conditions intended to defeat your design (for example a stolen unlocked phone, a corrupted sync mid-transfer, or a Class C device with no secure storage). Before applying them:

1. analyse why each condition is challenging.
2. Predict how your design will behave.

Then apply them and determine whether the design still meets the requirements, which decision is critical, and whether any assumption in your design fails. If the design survives, analyse why the conditions do not defeat it; if not, state the refinement required.

Evidence Expected: Student-designed adverse conditions, predictions, traces and conclusions.

Assessment Distribution

Task	Competency Assessed	Marks

Task 1	Diagnose and repair an architecture description	6
Task 2	Construct and apply a cross-platform architectural comparison	8
Task 3	Analyse runtime environments and application execution	9
Task 4	Diagnose and repair mobile security design	7
Task 5	Design and adapt power management policy	7
Task 6	Independently design a platform solution for an unseen problem	13

Task	**Competency Assessed**	**Marks**
Total		**50**

Submission Requirements

Your submission must contain the following evidence wherever applicable:

Scenario Interpretation → Components and Constraints → Architecture or Policy Decision → Structured Specification → Scenario Trace → Boundary or Adversarial Test → Interpretation

Important Instructions

1. This is an individual assignment.
2. Do not use an IDE, emulator or SDK to solve the tasks. The purpose is to demonstrate analysis and design before implementation.
3. Source code in Java, Kotlin, Objective-C, Swift, C#, C++ or any other language will not be accepted as a design specification.
4. All designs must follow the SIMATS Engineering guidelines for technical design documentation.
5. Avoid statements such as:
    - Use a secure OS
    - Optimise the battery appropriately
    - Choose a suitable platform
    - Apply best practices

Such statements hide the technical decision and will be considered incomplete.

6. Appropriate abstraction is permitted, for example "the kernel scheduler selects the next runnable process" or "the OS verifies the package signature", when the internal implementation of that operation is not being assessed.
7. A correct conclusion without the required reasoning, specification or trace will not receive full credit.
8. The Day-4 Design Challenge (Task 6) is compulsory.

ITA03-MOBILE COMPUTING ASSIGNMENT-4B

ASSIGNMENT 4 – PART II

MOBILE SYSTEMS ENGINEERING: ANALYSE, INSTRUMENT, TEST, DEBUG & OPTIMISE

Course Outcome: CO4 – Analyse mobile operating systems, architectures, runtimes, security and power management techniquesNature: IndividualDuration: One WeekTotal Marks: 50Environment: Android Studio / Emulator / Physical device with adb, Linux shell, Python or Kotlin/Java (student's choice)Prerequisite: Completion of CO4 – Assignment Part I

TASK 1 – ANALYSE AND VALIDATE A MOBILE OPERATING SYSTEM MODEL Inside

the OS: Android vs iOS as a Resource-Management Engine 8 Marks

A device manufacturer is selecting a mobile operating system for a new product line. The decision team requires evidence about how each OS organises its layers, schedules processes, manages memory and protects applications. Marketing statements are not acceptable evidence.

1.1 Analyse the OS Architecture | 3 Marks

Study the layered architecture of **Android** (Linux kernel, HAL, native libraries, Android Runtime, application framework, applications) and **iOS** (Core OS, Core Services, Media, Cocoa Touch). Using an emulator/device (adb shell, dumpsys, ps, /proc) or documented tooling, your report must show:

- Layer-by-layer responsibility mapping of both operating systems ● Process model: zygote/app-process creation (Android) versus launchd/app launch (iOS)
- Inter-process communication mechanism (Binder versus XPC/Mach ports) with at least one live evidence item
- Component lifecycle (Activity/Service/Broadcast Receiver versus UIViewController/UIApplication states)

Do **not** output only a comparison table copied from a textbook. Each row must be supported by an observed command output or a documented source.

1.2 Expose the OS Behaviour | 2 Marks

Instrument the system to display sufficient runtime state to demonstrate how the OS manages applications. For example: running process list with oom_adj / process state, *dumpsys meminfo, dumpsys activity processes*, or lifecycle callback logs from a test application.

analyse what each displayed value means and how it influences the decision to keep or kill a process.

1.3 Requirement Change | 2 Marks

Run the same test application under at least three different memory/foreground conditions (e.g. foreground, background for 5 minutes, low-memory emulation using *am send-trim-memory* or a reduced-RAM AVD). Before executing each case, predict:

- Whether the process will remain alive, be trimmed, or be killed ● Which lifecycle callback will be the last one executed
- Whether the app's saved state will be restored or lost on return Compare your

predictions with the actual logs and interpret every mismatch. **1.4 Boundary Test | 1 Mark**

Design one boundary case that could expose a lifecycle or state-management defect (for example screen rotation during an active background task, or process death while a dialog is open). analyse why you selected it.

Evidence Expected

Commands executed, logs/screenshots with relevant lines highlighted, lifecycle trace, predictions, modified cases and interpretation.

TASK 2 – ARCHITECTURE ASSUMPTIONS vs MEASURED REALITY Build It,

Break It, analyse It

9 Marks

A company is designing a sensor-driven delivery-tracking application for a mid-range smartphone built on a heterogeneous multi-core System-on-Chip (for example big.LITTLE / DynamIQ: high-performance cores plus efficiency cores, GPU, DSP/NPU and a modem).

Implement:

Solution A – A single-threaded (main-thread) implementation of a compute-heavy task (e.g. image filtering or route-data parsing)

Solution B – A multi-threaded / background-executor implementation of the same task (Kotlin coroutines, ExecutorService, or WorkManager)

Your implementation must report measured evidence — not merely that the task completed.

2.1 Normal Case | 2 Marks

Create a workload on which the multi-threaded design produces a clear improvement. Show frame timing / task duration, the thread-to-core placement (e.g. *top -H*, systrace/Perfetto, or CPU profiler), and the application-level architecture (MVVM / MVP layers) used to keep the UI responsive.

2.2 Break the Assumption | 3 Marks

Construct your own workload for which the intuition "more threads = faster and better" fails — for example a very small task where thread-creation and synchronisation overhead dominates, a lock-contended workload, or a task that causes UI jank or ANR because work was moved incorrectly.

Your program must demonstrate:

Multi-threaded Result is NOT better than Single-threaded Result (time, jank frames or energy).

Identify the first design decision that causes the degradation.

2.3 Engineering Comparison | 2 Marks

Instrument both solutions sufficiently to compare:

- Computational work (CPU time, thread count, context switches) ● Memory requirement (heap, PSS, allocations)
- User-perceived quality (frame drops, response latency)
- Predictability / worst-case guarantee

Do **not** rely on **elapsed time** alone.

2.4 Changed Requirement | 2 Marks
Management now states:

"The application must run for a full working day on a single charge. Slightly slower processing is acceptable."

Determine how this requirement affects your core-placement and threading strategy (for example moving

work to efficiency cores, batching, or deferring through WorkManager). Your conclusion must be based on the behaviour of your implementations, not a generic textbook statement.

Evidence Expected

Two implementations, profiler/trace evidence, student-designed counter-example, resource-usage measurements and an engineering interpretation.

TASK 3 – RUNTIME BEHAVIOUR UNDER CHANGING EXECUTION CONDITIONS

Interpreter + JIT + AOT + Garbage Collection + Cross-Platform Runtimes 10 Marks

You are developing the runtime-performance component of a mobile platform evaluation. Create a benchmark application containing at least **three** workloads: (i) a CPU-bound loop/algorithm, (ii) an object-allocation-heavy routine and (iii) a UI-list rendering scenario.

Stage A – Normal Execution

Run the benchmark on the Android Runtime (ART). For the selected workloads, display:

- Cold-start time and warm-start time
- Compilation state of the application (*dumpsys package* / *cmd package compile* output)
- Garbage-collection events and pause durations (logcat GC lines or profiler) ● Heap growth over the execution of the benchmark

Stage B – Change the Compilation Mode

Force a different compilation filter (e.g. *verify* / interpret-only, *speed-profile*, *speed*). Before executing each modified run, predict:

- Which workload will be affected most
- Whether start-up time, steady-state speed or storage footprint will change, and in which direction

Now run the modified cases and compare. Your test must be deliberately designed to expose a meaningful difference where possible; simply switching a flag on an irrelevant workload is insufficient. analyse the roles of interpreter, JIT, profile-guided AOT and Baseline Profiles in your result.

Stage C – Production Failure

Introduce a reachable runtime failure: a deliberate memory leak (e.g. a static reference to an Activity or an unbounded cache) and a main-thread blocking call. Your program/instrumentation must detect and report the condition (heap growth trend, *OutOfMemoryError*, GC thrashing, ANR trace, LeakCanary or StrictMode violation) rather than silently continuing. analyse how the detection works and which runtime component (heap manager, garbage collector, watchdog) reports it.

Stage D – Change the Requirement

The product team now wants one code-base for Android and iOS. Compare, using your own

measurements or documented runtime architecture:

- Native ART (Kotlin/Java) and Swift with ARC (compiled, reference-counted) ● Flutter (Dart AOT with its own rendering engine)
- React Native / WebView-based hybrid (JavaScript engine with bridge)

interpret start-up, memory management model, UI-thread behaviour and access to native APIs. Recommend a runtime for the product and justify it with your Stage A–C evidence.

Evidence Expected

Benchmark source code, compile-mode commands, GC/heap logs, prediction before modification, failure-detection evidence, cross-platform comparison and interpretation.

TASK 4 – SECURE MOBILE APPLICATION AUDIT

Threat Modelling, Sandboxing and Permission Enforcement

7 Marks

A healthcare start-up is releasing a mobile application that stores patient appointment data, uses the camera and location, and communicates with a REST back-end. The
security team needs a system that determines: **what can be attacked, which protection mechanism stops the attack, and what remains exposed.**

Task 4.1 – Build the Security Audit Model | 3 Marks

Develop (or script) a security audit for a small test application. Your audit must accept:

- List of requested permissions (manifest / Info.plist)
- Data stores used (SharedPreferences, SQLite, files, Keystore/Keychain) ● Network endpoints and protocols used
- Exported components / URL schemes

The audit must determine and display:

- Threat model (STRIDE or equivalent) with at least six identified threats ● Protection mechanism for each threat (application sandbox / UID separation, SELinux, runtime permissions, app signing, verified boot, Keystore/Secure Enclave, TLS with certificate pinning)
- Residual risk after the mechanism is applied
- Final security verdict for the application

For example, the output should make the chain understandable:

Threat: Local data theft from a rooted device

↓

Mitigation: Encrypted storage + hardware-backed Keystore key

↓

Residual risk: Key extraction on a compromised OS – accepted / escalated Returning only a

list of permissions is **not sufficient**.

Task 4.2 – Expose the Decision Process | 1 Mark

Instrument your audit to display the security decision for each asset. For each asset, show the attack vector, the mechanism that blocks it, and whether it was Blocked / Partially Blocked / Exposed.

Evidence Expected

A trace such as:

Asset	**Attack Vector**	**Mechanism Applied**	**Result (Blocked / Partial / Exposed)**	**Reason**
Auth token	...	...	...	...
Local DB	...	...	...	...
API traffic	...	...	...	...
Exported activity	...	...	...	...

The objective is to make defence-in-depth visible through execution.

Task 4.3 – Predict Before You Execute | 1 Mark

Select one security control and weaken it deliberately (for example set *android:exported="true"* on an internal component, allow cleartext traffic, or store a token in plain SharedPreferences). Before running the modified application, predict:

- Which asset becomes exposed
- Which attack path is most likely to succeed
- Whether the audit verdict will change

Record your prediction first. Then execute the audit/attack demonstration (e.g. *adb shell am start*, *run-as*, proxy interception on your own test app) and compare the observed behaviour.

Task 4.4 – Security Incident Challenge | 1 Mark

A third-party library used by the application is found to leak the device identifier over an unencrypted channel. Determine:

- Which asset and which layer of the OS security model is affected ● Whether the sandbox or permission system can contain the leak ● At which point the leak can be detected (static analysis, network capture, runtime hook)
- What change restores the security guarantee

analyse why your remediation is sufficient.

Task 4.5 – Engineering Extension | 1 Mark

Management proposes adding biometric login and an enterprise work-profile (BYOD) mode. Do **not** implement it. Instead analyse what would have to change in your **threat model, trust boundaries and audit logic** (key attestation, user-authentication binding, data separation between profiles, remote wipe). Your answer should demonstrate that you understand the underlying security model rather than a list of features.

Evidence Expected

Working audit, permission/manifest evidence, threat-to-mechanism trace, prediction before-execution evidence, incident experiment and reasoned extension.

TASK 5 – THREE POWER ENGINEERS, ONE DEVICE

DVFS + Doze/App Standby + Wakelock & Batching

7 Marks

A company must extend battery life of a field-service smartphone while keeping its tracking features usable. Use the same test application and the same device/emulator profile (Battery Historian, *dumpsys batterystats*, Android Studio Energy Profiler, or a documented power model) and examine:

- **Technique 1:** CPU frequency/voltage control (DVFS, CPU governors, big.LITTLE core selection)
- **Technique 2:** OS-level idle control (Doze mode, App Standby Buckets, background execution limits)
- **Technique 3:** Application-level control (wakelock discipline, JobScheduler/WorkManager batching, network request coalescing, sensor batching)

Do Not Compare Only Total Battery Percentage

Instrument each technique to display the sequence of power-state changes. For each technique determine:

- What event or condition triggers the technique
- What system information it must maintain (utilisation, screen/motion state, app bucket, job

constraints)

- How power consumption changes over time after the technique engages ● Measured effect (wake-up count, wakelock time, radio-active time, estimated mAh)
- Side-effect on responsiveness or functionality

Stress the Design

Modify the application to create a situation where two techniques conflict or one nullifies the other (for example a held partial wakelock preventing Doze benefits, or frequent exact alarms defeating batching).

Determine:

Can a power-saving technique at one layer be defeated by behaviour at another layer? analyse your

observation.

Industry Decision

Consider three deployment environments:

- A. Always-on location/sensor tracking device (continuous background work) ● B. Media/gaming-intensive device (sustained high CPU/GPU load) ● C. Large fleet of managed devices with mixed OS versions and enterprise policies

interpret which characteristics of the three techniques matter when selecting a power strategy for each environment. Do **not** provide a selection based only on memorised definitions; connect your reasoning to observed behaviour, scheduling constraints and the hardware components that consume power (CPU, radio, display, GPS).

Evidence Expected

Three instrumented measurements, power-state traces, modified (conflicting) application, stress-case evidence and deployment-oriented analysis.

TASK 6 – BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE

Integrated Mobile System Design (Unseen Optimisation Problem) 9 Marks

A logistics company equips delivery staff with smartphones. The tracking application samples location and sensor data, stores it locally, and uploads it to a server. The vehicle must keep working for a full shift while data stays protected.

The objective is to:

Design a system that maximises useful tracking data delivered per unit of battery energy while keeping the data secure.

Task 6.1 – Model the Problem | 1 Mark

Clearly identify:

- Input
- Output
- Objective
- Constraints

analyse what information must be retained at an intermediate stage (battery level, charging state, network type, last upload time, queued records, device temperature) to distinguish two situations that look identical on the surface but need different decisions.

Evidence Expected

A concise model showing why the battery level alone is insufficient to decide when to sample or upload.

Task 6.2 – Design the Architecture and Policy State | 2 Marks Define the

system as a state-based policy of the form:

Policy[BatteryState][NetworkState][MotionState] → { SamplingRate, UploadAction }

analyse precisely:

- What each state dimension stores
- What the initial/default state is
- How the system moves from one state to another (transition conditions) ● How security (encryption before queueing, authenticated upload) is incorporated in every transition

Draw the layered architecture (UI, domain, data, OS services) showing where each component runs (main thread, background executor, foreground service, WorkManager job).

Evidence Expected

State definition, default condition, transition rules and architecture diagram.

Task 6.3 – Implement the Prototype | 2 Marks

Implement the prototype (on a real/emulated device or as a faithful simulation). Use a dataset/scenario containing at least **6 distinct operating conditions** (e.g. charging on Wi-Fi, 80% battery on LTE, 20% battery stationary, low-power mode, no connectivity, device hot). Your program must display:

- Policy chosen for each condition
- Sampling interval and upload decision
- Estimated energy used per cycle
- Whether data was encrypted before storage and transmission For example:

Condition 3: Battery 20%, stationary, LTE

→ Sampling every 120 s | Upload deferred to charging/Wi-Fi window → Data encrypted

(AES-GCM, Keystore key) | Estimated energy = ... mAh Returning only a final energy

figure is **not sufficient**.

Evidence Expected

Source code, scenario table, chosen policies and corresponding energy estimates.

Task 6.4 – Validate the Solution | 1 Mark

For a smaller scenario containing 3 operating conditions, independently determine the expected behaviour by hand calculation, or by comparing against an always-on baseline (continuous sampling and immediate upload). Compare the result with the output produced by your prototype.

Answer: How does this validation increase your confidence that the policy logic is correct?
Evidence Expected

Independent reference result, prototype result and comparison.

Task 6.5 – Investigate Scalability and Load | 1 Mark

Instrument your prototype to record the number of policy evaluations, wake-ups, and queued records. Run the system for increasing fleet or event sizes, for example 100, 1,000, 10,000 records or 1, 5, 10, 20 simulated devices — as far as reasonably supported by your system.

Record:

Load (records / devices)	**Policy evaluations**	**Wake-ups**	**Memory / battery observation**
100			
1,000			
10,000			
...			

Do **not** rely on **execution time** alone.

Inquiry

analyse: Why does batching eliminate substantial repeated radio wake-ups but still not make continuous high-frequency tracking cheap in energy terms? Relate your explanation to the fixed cost of waking the radio/CPU and the amount of data collected.

Task 6.6 – Business Requirement Change | 1 Mark

The company now operates **5,000 devices** with several OS versions and requires real-time (within a few seconds) location updates during emergencies.

Management states:

"Reliable delivery matters more than battery during an emergency."
Do **not** implement this change. Based on the behaviour observed in Task 6.5:

- analyse why your current battery-saving policy becomes unsuitable in this situation.
- State what property of the requirement has now changed.
- Propose the type of strategy you would investigate (e.g. adaptive/priority-based policy, foreground service with high-priority push, edge/cloud offloading, server side fleet management).
- analyse the trade-off the company would be accepting (energy, heat, privacy exposure, cost).

Evidence Expected

A reasoned engineering response connecting energy behaviour with the changed business requirement.

Task 6.7 – Technical Reflection | 1 Mark

In not more than 150 words, answer:

"A power-saving technique that works well in one scenario of your prototype may fail in another. Why does this mean that no single mobile OS or power management feature is optimal for every application?"

Your explanation should refer to evidence obtained from your own implementation.

MARK DISTRIBUTION

Task	Engineering Competency	Marks
1	Mobile OS analysis & lifecycle validation	8
2	Architecture analysis & adversarial profiling	9

3	Runtime analysis & failure handling	10
4	Security auditing, debugging & regression testing	7
5	Power management implementation & engineering comparison	7
6	Unseen integrated optimisation prototype	9
	TOTAL	**50**

MANDATORY SUBMISSION EVIDENCE

For each applicable task, submit:

Source Code → Test Input → Expected Behaviour → Actual Output → Intermediate System Evidence (logs / traces / profiler data) → Boundary/Failure Test → Interpretation

Screenshots containing only a final output such as:

Battery Drain = 12% / hour

will **not** constitute sufficient evidence.

SOFTWARE ENGINEERING EXPECTATIONS

Your implementation will also be examined for:

- Meaningful variable and function names
- Modular design (UI, business logic and data layers separated) ● Appropriate data structures and Android/iOS APIs
- Separation of core logic from input/output and instrumentation code ● Handling of boundary conditions (low memory, no network, permission denied, process death)
- Clear assumptions (device model, OS version, emulator settings) ● Reproducible test cases and commands
- Correct handling of invalid or exceptional system conditions where relevant

A program that produces the expected output but whose logic cannot be analyseed or validated will **not** receive full credit.

IMPORTANT RULE

DO NOT STOP WHEN THE APPLICATION RUNS.

A successful build and launch establishes only that the application is installable. You must

establish:

Does it behave correctly? → When does it fail? → Why does it work? → What happens when the requirement or the device condition changes?

ENGINEERING WORKFLOW

DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND

ASSIGNMENT 4 –ANSWERS

Mobile Computing / Mobile Application Development Environment | CO4

Formatting note: the answers are placed after the original questions. Tables use black borders, white backgrounds and black text. For Part II, any sample measurements or logs below are illustrative examples, not claimed measurements from a real device; replace them with your own screenshots, commands and profiler results before submission.

PART I – ARCHITECTURAL ANALYSIS AND DESIGN

TASK 1 – Architecture Review (6 Marks)

1.1 Diagnose the Description

No.	Technical deficiency	Why it prevents reliable implementation/evaluation
1	Layers are omitted: device drivers, HAL, native libraries and system services are not identified.	Engineers cannot determine which component translates hardware events, exposes APIs or implements framework services.
2	The kernel is described as running applications.	The kernel schedules processes/threads, manages virtual memory, files, networking and device access; user applications normally execute in user space.
3	Kernel family and structure are unspecified.	Linux-based Android, Darwin/XNU-based iOS and Windows NT have different interfaces, drivers and security boundaries.
4	Runtime behaviour is vague.	It does not say whether delivered code is native machine code or bytecode, who loads it, or whether interpretation, JIT or AOT compilation is used.
5	Permissions are treated as a generic security solution.	It omits install-time/runtime consent, OS enforcement points, sandboxing, signing and privileged-service checks.

6	Power saving has no trigger or mechanism.	No idle state, CPU frequency scaling, Doze/background policy, wake-lock rule or exit condition is specified.
7	Application-to-OS communication is missing.	The note does not describe framework APIs, system-service IPC, Binder/XPC/Windows IPC or kernel system calls.
8	No lifecycle/process model is defined.	Background execution, process death, state restoration and callback order cannot be predicted.

1.2 Repaired Android Layered Architecture

Layer (top to bottom)	**Responsibilities**	**Interface exposed upward**	**Depends on**
Applications	UI, app logic, local state; runs in an app sandbox.	Android framework APIs and app components.	Application framework; app process/runtime.
Application framework and system services	Activity Manager, Package Manager, Notification Manager, Location Manager, permissions and lifecycle.	Java/Kotlin APIs and service calls; Binder IPC for service requests.	ART, native libraries, Binder and kernel services.
Android Runtime (ART)	Loads DEX bytecode, verifies it, interprets/JIT-compiles and uses profile-guided AOT where applicable; manages heap and garbage collection.	Runtime execution, class loading, JNI and managed memory.	Linux process/memory/thread primitives and native libraries.
Native libraries	C/C++ libraries for graphics, media, database,	Native APIs used by framework/runtime through JNI or native interfaces.	HAL, drivers and kernel interfaces.

	cryptography and networking.		
Hardware Abstraction Layer (HAL)	Standard interfaces for camera, audio, sensors and other vendor hardware.	Stable hardware service interfaces to Android services.	Vendor implementations and device drivers.
Linux kernel	Process/thread scheduling, virtual memory, networking, IPC primitives, permissions and device access; SELinux enforcement and cgroups support resource controls.	System calls, Binder driver, device nodes and kernel interfaces.	CPU, memory, buses and physical devices.
Hardware	CPU, RAM, storage, radio, display, sensors and peripherals.	Physical interrupts, registers and buses.	Physical power and device design.

Explicit locations: kernel = Linux kernel layer; runtime = ART; security enforcement = app UID sandbox, SELinux, permission checks in framework/system services, package signature verification and Keystore-backed key operations; power management = Android power manager, Doze/App Standby and kernel CPU/resource controls coordinated across framework and kernel.

1.3 Design Defence (under 100 words)

This specification is precise because each layer has a defined responsibility, an interface exposed upward and an explicit dependency downward. It identifies the kernel family, runtime, IPC path, security enforcement points and power-management components. An engineer can trace a request from an application API through a system service and kernel interface to hardware, then identify where access is checked. The description is testable: process state, permissions, Binder calls, compilation state and power behaviour can be observed. It therefore specifies architecture and boundaries rather than making broad promotional claims.

TASK 2 – Android vs iOS vs Windows (8 Marks)

2.1 Requirement Model

Item	**Definition**

Input	Platform architecture details: kernel, runtime, process lifecycle, IPC, permissions, background delivery and power controls.
Output	Comparison matrix and a justified platform-specific background-message design.
Objective	Deliver incoming messages reliably while preserving privacy, responsiveness and battery life.
Constraints	OS background limits, terminated-process behaviour, network availability, push-service dependence, encryption and user permissions.
Decision	Select a platform-specific delivery strategy and identify features that must differ by OS.
Information needed	Kernel family, app process model, runtime/code format, notification/push service, background task rules, sandbox, IPC and battery restrictions.

2.2 Hypothesis

The operating system's background-execution and push-notification policy will affect message delivery more than the language runtime alone, because the OS can suspend or terminate an application even when its code is correct.

2.3 Comparison Matrix

Feature	**Android**	**iOS**	**Windows**	**Messaging consequence**
Kernel family/structure	Linux kernel; monolithic kernel with loadable modules and Android-specific IPC/security additions.	Darwin/XNU hybrid kernel combining Mach and BSD components.	Windows NT hybrid kernel architecture.	Kernel family affects drivers, process scheduling and system interfaces, but app background policy is also decisive.
Kernel-level interface	Linux system calls, Binder driver and device interfaces;	Darwin/BSD system interfaces and Mach	Windows NT system calls are generally accessed through	Use supported APIs; direct kernel interfaces

	apps usually use framework APIs rather than direct kernel calls.	mechanisms; apps use public frameworks.	supported Win32/WinRT APIs.	are not a portable app strategy.
Runtime	ART executes DEX; interpreter/JIT plus profile-guided AOT compilation.	Swift/Objective-C compiled to native code; Objective-C runtime supports dynamic dispatch; Swift uses ARC.	WinRT is an API/runtime projection; native code and .NET implementations depend on app technology.	Runtime affects launch, memory and API access, but does not guarantee background execution.
Framework	Android SDK, Activity/Service, NotificationManager, WorkManager and FCM integration.	Cocoa Touch/UIKit/Swift UI, UserNotifications and APNs.	WinRT/UWP or supported Windows app frameworks, notifications and background tasks subject to version/policy.	Use each platform's approved notification and background-task mechanism.
Isolation	Per-app Linux UID sandbox, SELinux, permissions and scoped storage.	Per-app sandbox, entitlements, code signing and privacy permissions.	AppContainer/package identity and capability-based access for packaged apps.	Store tokens in protected storage and minimise permissions on every platform.
IPC	Binder between app and system services; intents and content providers at framework level.	XPC and Mach ports for system/app services; URL schemes and extensions for app-level integration.	COM/WinRT APIs, app services and supported IPC mechanisms.	IPC is OS-specific; abstract the messaging domain behind a shared interface.
Background delivery	Push through FCM; notifications and eligible work; exact timing depends on priority, restrictions and network.	APNs remote notifications; background execution is tightly controlled and not guaranteed as arbitrary continuous execution.	Push/notification and background-task capabilities depend on Windows edition, app model and policy.	Use server-side message queue, push trigger, local durable storage and retry/idempotency.

2.4 Desk-check: incoming message with screen off

Step	**Android trace**	**iOS trace**

1 Network event	Message reaches server; server sends push through FCM.	Message reaches server; server sends push through APNs.
2 Radio/kernel	Modem/network stack receives traffic; kernel network stack and driver process it.	Modem/network stack receives traffic; Darwin networking and driver process it.
3 System push service	Google Play services/FCM delivers push to OS/app pathway, subject to device configuration.	APNs and iOS notification service deliver push according to notification/background policy.
4 App/runtime	If eligible, Android schedules app callback or notification handling; ART executes code in app process.	If granted and eligible, iOS wakes the app briefly for background notification handling; native runtime executes callback.
5 App action	Validate payload, persist message transactionally, post notification; defer large sync to WorkManager.	Validate payload, persist message, post notification; defer work within permitted background time.
6 Result	Message may be visible through notification even if full app work is deferred.	Notification may be displayed by OS; background processing is limited and not guaranteed for every push.

2.5 Challenge: app process was terminated

Prediction: delivery and execution may differ. Android high-priority FCM can cause eligible delivery, but restrictions, force-stop, OEM policy and quota can prevent immediate app execution. iOS may show a user-visible notification without running arbitrary app code; silent pushes can be delayed or discarded. Responsible elements are push-service policy, app state, permissions, quotas and OS scheduler—not simply ART versus Objective-C runtime. Trace: push service receives event → OS checks app state/priority → either posts notification or schedules a short callback → app persists data if launched. Compare actual logs on the chosen OS versions; the result is configuration-dependent, so no universal guarantee is claimed.

TASK 3 – Runtime Environment Analysis (9 Marks)

3.1 Execution Models

Platform	**Delivered code / loader**	**When executable machine instructions are available**
Android / ART	APK contains DEX bytecode and resources; Package Manager installs package; ART/runtime loads DEX.	Interpretation/JIT may execute/compile at runtime; profile-guided AOT can compile selected code ahead of time. Exact state depends on Android version and compilation filter.
iOS / Objective-C runtime	App is signed native Mach-O executable; dyld loads executable and libraries; Objective-C runtime resolves classes/selectors.	Native machine code is produced before delivery; dynamic linking and Objective-C dispatch occur during load/runtime.
Windows	Depends on app model: native PE binary or managed .NET IL; Windows loader and relevant runtime load it.	Native code executes as machine instructions; managed code is JIT/AOT compiled according to framework and deployment model. WinRT is an API/runtime model, not one universal compiler.

3.2 Android launch sequence

Order	**Component/action**	**Precondition and purpose**
1	Launcher receives tap and requests app launch through system services.	Installed package and launcher entry must exist; translates user action into launch request.
2	Activity Manager checks process/activity state and launch permissions.	System service is running; determines whether to reuse or create a process.
3	Zygote forks app process when a new process is needed.	Zygote and system resources already exist; process creation shares preloaded state efficiently.

4	Linux kernel allocates/schedules process and maps memory.	Kernel scheduler, virtual memory and app UID security context are available.
5	ART initialises, loads DEX/classes and performs required verification/linking.	App process and runtime libraries exist; prepares managed code execution.
6	Application object and Activity lifecycle callbacks run.	Framework services and runtime are ready; app initialises state and inflates/creates UI.
7	UI thread lays out, draws and submits frames to graphics stack.	Resources and window surface are available; first screen becomes visible.

3.3 Runtime strategy comparison

Dimension	**Android ART**	**iOS Swift/Objective-C**	**Windows**
Interpret/JIT/AOT	DEX; interpreter and JIT; profile-guided AOT; Baseline Profiles can improve common startup paths.	Swift/Objective-C code is compiled to native code before distribution; Objective-C retains dynamic runtime dispatch; no general Dalvik-like app bytecode interpreter.	Native apps use native binaries; .NET apps use managed IL with runtime compilation/AOT options depending on app model.
Memory	Managed heap with tracing garbage collection; native allocations also need explicit ownership management.	Swift ARC uses reference counting; Objective-C objects generally use ARC in modern apps, with runtime/dynamic dispatch.	Native ownership depends on C/C++; .NET uses garbage collection.
Responsiveness/battery	GC pauses can occur; JIT costs CPU temporarily, AOT can improve repeated	ARC updates reference counts during object ownership changes; cycles need	Managed GC may pause; native code has manual/resource ownership risks;

	execution but increases storage.	weak/unowned references; native code avoids managed GC pauses for Swift objects.	behaviour varies by framework.
Native access	JNI and NDK libraries cross from managed ART code to native libraries.	Public Apple frameworks and C/Objective-C interoperability expose native APIs.	WinRT/Win32 APIs and language projections expose OS functions.
Startup/memory	Cold start includes process/runtime/class initialisation; warm start can reuse process/cache; compiled code can increase storage.	Native code avoids bytecode JIT at launch, but dynamic linking and framework initialisation still cost time/memory.	Depends on native vs managed stack, package size and runtime initialisation.
Portability	Java/Kotlin source and Android APIs target Android; native libraries are ABI-specific.	Swift/Objective-C and Apple frameworks are tied to Apple platforms.	Windows APIs and .NET portability depend on target framework and app model.

Low-memory conclusion: no runtime is universally best. For a tightly constrained device, native compiled code with a small footprint may be attractive, but memory use depends more on app architecture, allocation patterns and libraries than runtime label alone. ART can work well with bounded allocations and measured GC behaviour; ARC can reduce tracing-GC pauses but reference cycles and object churn still cost memory/CPU. Choose using measurements on the target device.

3.4 Cold and warm start desk-check

Step	**Cold start**	**Warm start**
Process creation	Usually required if process is absent.	Skipped if existing process remains alive.
Runtime/class initialisation	Most required initialisation occurs.	Often reused; some lazy initialisation may still occur.
Activity creation	Create Activity and restore saved state if available.	Activity may be recreated or resumed depending on lifecycle state.

Resource/UI creation	First window/layout/rendering path executes.	Existing views/state may be reused or redrawn.
Reason for difference	No resident app process or warm caches.	Resident process and cached classes/resources reduce work; OS may still kill it at any time.

3.5 Dependency analysis

The kernel must create and schedule a process and provide memory before ART can initialise. ART must load classes and prepare execution before application code can safely run. Framework services must be available before lifecycle callbacks can request windows, permissions or notifications. If these dependencies are reversed, callbacks could run without a process, memory, runtime or service endpoint. The launch sequence is therefore a dependency chain, not merely a list of events.

TASK 4 – Security Forensics (7 Marks)

4.1 Defects and mechanisms

Original decision	**Bypassed mechanism**	**Risk / why OS cannot fully protect**
Request every permission at install for future use.	Least privilege and runtime permission consent.	User may grant unnecessary access; OS permission checks cannot infer that app logic does not need the data.
Save login token in file readable by other apps.	App sandbox, UID separation and secure storage.	Another app that can read the file can steal the token; sandboxing is undermined by placing it in shared storage.
Accept updates from any source without verifying producer.	Package/code signing and trusted update channel.	Malicious code can replace app or components; OS signing checks help only when update/install trust is enforced.
Use unencrypted server connections.	TLS transport protection and server authentication.	Network observer can read or alter tokens/data; OS permissions do not encrypt application traffic.

4.2 Attack scenario

A malicious app obtains access to a shared location where the token was stored. It reads the token file; confidentiality is first lost at the file-read step.

The attacker uses the stolen token to call the SecureNotes API from another device. If the server accepts the token without additional binding or revocation, account data is exposed.

Separately, an attacker on public Wi-Fi observes plain HTTP traffic and captures a later login/session token. Confidentiality is lost when the unencrypted token crosses the network; an active attacker could also modify responses.

4.3 Minimal repairs (Android example)

Repair	**OS mechanism and placement**
Request only necessary permissions at the point of use.	Android runtime permission APIs and framework permission checks; explain purpose and handle denial.
Store tokens in app-private storage and encrypt sensitive data using a key protected by Android Keystore.	Linux UID sandbox/SELinux protect private files; Keystore limits key extraction; avoid putting secrets in shared files or logs.
Accept updates only from the trusted distribution channel and verify package signing identity.	Package Manager verifies package signatures; use controlled release/update distribution and protect signing keys.
Use HTTPS with certificate validation; consider pinning only with a safe rotation plan.	TLS is provided through platform networking/crypto libraries; server identity is validated before credentials are sent.

4.4 Regression check

Legitimate behaviour: after the user explicitly grants camera permission, the app can still attach a note photo. The app requests CAMERA only when the feature is used, checks the result, and handles denial. Token storage and HTTPS changes do not remove this feature, so the regression test is: grant permission → capture photo → upload over HTTPS → confirm the note is saved; then deny permission and verify the app remains usable without camera access.

TASK 5 – Power Management Reasoning (7 Marks)

5.1 State-transition policy

State	**Hardware/kernel behaviour**	**Entry condition**	**Exit condition**	**Permitted work**
ACTIVE	Display may be on; CPU uses performance level appropriate to load;	User interaction, visible UI or time-critical operation.	No interaction and no urgent task; transition to IDLE.	Foreground UI, sensor sampling at required rate,

	kernel schedules foreground work.			short bounded computation.
IDLE	Display off; CPU can use lower frequency/idle states; network/background work restricted by OS.	Screen off and no active user interaction; app has no justified wake lock.	User interaction, scheduled job or push event.	Batched sensors, scheduled jobs, eligible push handling; no permanent busy loop.
SUSPENDED/DOZE	CPU mostly asleep; wakeups and network maintenance limited; hardware may retain low-power state.	Device stationary/unplugged with screen off, subject to OS policy.	Motion, charging, user interaction or permitted high-priority event.	OS-controlled alarm/push handling and short bounded work; avoid continuous polling.

5.2 Urgent push while suspended

The insufficient assumption is that the app must keep the CPU permanently awake to receive every event. Push-based wake-up lets the OS/network push service wake the device when appropriate, while Doze may defer ordinary work. The app should remove its permanent partial wakelock, process only the urgent event in a bounded window, persist it, and schedule non-urgent processing through WorkManager/JobScheduler. Wake locks are justified only for a short task that truly must keep the CPU running.

5.3 Battery saver at 15%

Level	**Policy at 15% charge**
Kernel/hardware	Reduce performance frequency/voltage where supported; prefer efficiency cores; reduce unnecessary wakeups and background CPU activity.
Runtime/framework	Apply OS battery-saver/background restrictions; defer non-urgent jobs; reduce animation/rendering and optional polling.
Application	Increase non-urgent sensor interval, batch location/network uploads, stop repeated retries, persist queue and upload when charging or suitable connectivity returns.
Trade-off	Location refresh may be slower and non-urgent sync delayed. This is acceptable

	because core data remains queued securely and urgent events still use the permitted priority path.

TASK 6 – RuralCare field-data solution (13 Marks)

Stage 1 – Interpret the problem

Item	**Specification**
Input	Patient forms, timestamps, device class, connectivity, battery, permissions and local queue.
Output	Protected local records synchronised exactly once logically to the server, with an audit trail.
Objective	Complete a ten-hour shift offline-capably without losing or duplicating records and protect patient data if a device is lost.
Constraints	Android/iOS smartphones and J2ME/WML feature phones; long offline periods; limited battery/storage; differing encryption and background APIs.
Decision	Use native smartphone clients plus a constrained legacy fallback, shared server API and idempotent sync protocol.

Stage 2 – Platform strategy

Use native Android and iOS clients for Classes A and B because they provide stronger app sandboxing, secure storage, modern encryption and OS-managed background work. For Class C, use a minimal J2ME client with local record queue if persistent storage is available; use WML only as a connected fallback, not as the sole offline workflow. If a feature phone cannot encrypt and protect offline patient records, do not store identifiable records on it; use a minimal coded/tokenised form or restrict that device class from sensitive offline collection. This is a security requirement, not a convenience choice.

Stage 3 – Constraints and retained sync information

Class	**Runtime/security**	**Power/storage/connectivity constraints**	**Design decision**

A Android	ART; UID sandbox, runtime permissions, Keystore, TLS.	SQLite/Room encrypted database or encrypted fields; WorkManager; Wi-Fi/LTE; Doze.	Encrypt records before durable queueing; unique record IDs; sync worker.
B iPhone	Native Swift/Objective-C; sandbox, Keychain, data protection, TLS.	Core Data/SQLite protected storage; background task limits; Wi-Fi/cellular.	Use Keychain-protected keys and OS-permitted background sync.
C feature phone	J2ME MIDP/CLDC or WML browser; security varies greatly by handset.	Small storage, limited cryptography and battery; intermittent GSM/data; WML usually requires connection.	Use signed J2ME where possible and minimal encrypted/tokenised data; if secure offline storage is unavailable, do not persist identifiable records.

Retained field	**Why it is required**
Unique record UUID	Makes retries idempotent and prevents duplicate server records.
Device/worker ID and schema version	Identifies source and interprets old queued records safely.
Creation time and sequence number	Preserves order and helps reconcile records created offline.
Sync state, attempt count and last error	Resumes interrupted sync and diagnoses repeated failures.
Payload hash / server acknowledgement ID	Detects corruption and confirms durable server receipt.
Encryption/key version	Allows correct decryption/rotation and avoids storing raw keys with data.

Stage 4 – Complete solution specification

Layer	**Class A Android**	**Class B iOS**	**Class C legacy**

UI	Native Activity/Compose forms; validation.	SwiftUI/UIKit forms; validation.	Small J2ME form; WML only for online fallback.
Domain	Validate patient data, assign UUID, enforce workflow.	Same business rules and record UUID policy.	Minimal validation and coded fields.
Data	Encrypted Room/SQLite queue; key protected by Keystore.	Protected local database; key in Keychain; file protection.	Encrypted local store only if device crypto is credible; otherwise no sensitive offline cache.
OS/background	WorkManager constraints; foreground service only if justified.	BGTaskScheduler/background URLSession where eligible; APNs for push.	MIDlet lifecycle and limited scheduled/network behaviour; no assumption of reliable background work.
Transport/sync	HTTPS/TLS; authenticated API; batched upload.	HTTPS/TLS; authenticated API; batched upload.	TLS if handset/browser supports it; otherwise avoid sensitive payloads or restrict use.
Lost device	Revoke tokens, remote wipe via MDM where available, key invalidation and server account lock.	Remote wipe/MDM, revoke credentials, protect data with device passcode and file protection.	Remote revoke credentials; assume remote wipe may be unavailable; keep stored data non-identifying/minimal.

Synchronisation procedure

Capture and validate a record; create UUID, timestamp, schema version and sequence number.

Encrypt sensitive payload before writing it to the local durable queue; store only key references, not raw encryption keys.

When network and power policy permit, select a bounded batch of unsynchronised records.

Send batch over authenticated TLS; each request includes UUID and payload hash.

Server validates identity and schema, inserts by UUID with an idempotency constraint, then returns acknowledgement IDs.

Client marks only acknowledged records as synced in a local transaction. Do not delete records before acknowledgement.

If connection fails, keep unacknowledged records queued and retry with exponential backoff and jitter; reuse the same UUIDs.

On conflict, compare server acknowledgement/version and route conflicting edits to reconciliation rather than silently overwriting.

Power and failure policy

During a ten-hour shift, batch non-urgent sync, reduce sensor frequency when stationary, avoid polling loops and release wakelocks promptly.

If storage is nearly full, warn the worker, prioritise critical forms, and never silently discard unacknowledged records.

If permission is denied, show which feature is unavailable and allow permitted manual entry where possible.

If no network exists, queue encrypted records and show pending count.

If authentication expires, retain encrypted queue and require re-authentication before upload.

If Class C cannot provide secure offline storage, disable offline identifiable-data capture on that device.

Stage 5 – Desk-check: Class A offline six hours, battery 20%, then phone lost

Step	State/action	Exposure, loss and recovery
1	Worker records data offline; each record receives UUID and is encrypted before queueing.	Plain patient content is not intentionally stored; data remains on device as ciphertext.
2	Battery reaches 20%; policy lengthens non-urgent sampling and defers upload while preserving the queue.	No record is discarded; unsynced records remain pending.
3	Phone is lost before sync; worker/admin reports loss.	Local ciphertext may remain physically present, but access is protected by device lock, app sandbox and encryption key protection.
4	Admin revokes access tokens and initiates MDM lock/wipe if available.	Future server access is blocked; remote wipe depends on device connectivity and management enrolment.
5	Recovery decision	If key is hardware-backed and device remains locked,

		data exposure risk is reduced; however, unsynced records may be unrecoverable if the device cannot be recovered and no backup exists.

Conclusion: confidentiality is improved, but availability of the six-hour unsynced records cannot be guaranteed after physical loss. A stronger policy may use an encrypted, authenticated incremental backup when connectivity briefly returns, without deleting the local copy until server acknowledgement.

Stage 6 – Attack your own design

Adverse condition	**Prediction and trace**	**Result/refinement**
Stolen unlocked smartphone	Attacker opens the app before lock/re-authentication; app sandbox alone does not stop use through the legitimate UI.	Partially survives. Require app re-authentication for sensitive records, short idle lock, device passcode, encrypted storage and immediate token revocation.
Sync interrupted after server insert but before client acknowledgement	Client retries same UUID; server idempotency key detects existing record and returns same acknowledgement.	Survives if UUID uniqueness and server transaction are enforced; test duplicate retry.
Class C handset has no trustworthy secure storage	Offline plaintext records could be extracted after loss.	Fails confidentiality requirement; do not store identifiable offline records, use tokenised minimal data, or exclude handset from sensitive workflow.
Device storage fills during offline shift	New records cannot be persisted reliably.	Warn before full, reserve storage for metadata, never report success until durable write succeeds; provide controlled escalation.

Critical decisions are encryption before persistence, unique record IDs with server-side idempotency, and refusing sensitive offline storage on insecure feature phones. The design depends on device-lock and key-protection assumptions, which must be tested per model.

PART II – MOBILE SYSTEMS ENGINEERING: ANALYSE, INSTRUMENT, TEST, DEBUG AND OPTIMISE

Important evidence note: the assignment requests real device/emulator commands, logs, profiler traces and measurements. The following provides a reproducible test plan and illustrative output formats. Do not submit the example numbers as if they were collected from your device; run the commands and insert your own screenshots/results.

TASK 1 – Android vs iOS OS resource management (8 Marks)

1.1 Layer responsibilities and process/IPC model

Area	Android	iOS
Layer mapping	Applications → framework/system services → ART/native libraries → HAL → Linux kernel → hardware.	Applications → Cocoa Touch → Media → Core Services → Core OS/Darwin → hardware.
Process creation	Zygote preloads common classes/resources; Activity Manager requests a fork for a new app process; app has its UID/sandbox.	launchd participates in system process/service launch; app launch services create app process under sandbox and code-signing policy.
IPC	Binder driver and framework Binder interfaces; intents/content providers also provide higher-level component communication.	Mach ports and XPC provide kernel-supported/system-service IPC; public frameworks abstract details.
Lifecycle	Activity onCreate/onStart/onResume; background transition through onPause/onStop; Services and receivers subject to modern background limits.	UIApplication/scene lifecycle and UIViewController callbacks; background execution is controlled by OS.
Evidence command	`adb shell ps -A`, `adb shell dumpsys activity processes`, `adb shell dumpsys meminfo <package>`, `adb shell cat /proc/<pid>/status`.	Use Xcode Instruments, Console/device logs and documented Apple development tools; Android adb commands do not apply to iOS.

1.2 Instrument runtime state

```
adb shell ps -A
adb shell dumpsys activity processes
adb shell dumpsys meminfo <your.package.name>
adb shell cat /proc/<PID>/status
adb logcat -v time
```

Interpretation guide: PID identifies the process; RSS/PSS estimate resident memory (PSS apportions shared pages); process importance/oom adjustment influences which process Android may reclaim under memory pressure; lifecycle logs show callbacks but do not alone prove that saved state was restored. Capture command output with timestamp, OS version and app package. Replace <PID> and package placeholder with actual values.

1.3 Three memory/foreground conditions

Test	Prediction before run	Evidence to capture	Interpretation
Foreground	Process likely remains alive; visible Activity gets priority.	PID, meminfo, lifecycle log.	Foreground importance usually reduces kill likelihood, not an absolute guarantee.
Background for 5 minutes	Process may remain, be cached or have work limited; outcome depends on memory and OS version.	Before/after PID, dumpsys processes, callback logs.	onStop may occur; process can be killed later without a final callback.
Low memory / trim	Caches may be trimmed; process may be killed if pressure continues.	Trim callback, logcat, PID check, saved-state restoration.	Do not assume onDestroy executes after process death; durable state must be saved earlier.

```
# Example test command; use only on your own test application/device
adb shell am send-trim-memory <your.package.name> RUNNING_LOW
adb shell dumpsys meminfo <your.package.name>
```

1.4 Boundary test

Rotate the screen while a form contains unsaved data and a background upload is running. Rotation may recreate an Activity while the upload continues. Expected behaviour: view state is restored, upload is not duplicated, and lifecycle logs show recreation/resume. This exposes accidental Activity references, duplicate observers and state stored only in view fields. Record logs before and after rotation and verify record UUID and upload count.

TASK 2 – Architecture assumptions vs measured reality (9 Marks)

2.1 Solution A and B design

Property	**Solution A: main thread**	**Solution B: background executor**
Work placement	CPU-heavy parsing/filtering runs on UI/main thread.	Worker/executor performs computation; main thread only updates UI.
Expected normal behaviour	Can delay touch response and cause jank/ANR for long work.	UI remains responsive; elapsed task time may improve if work is parallelisable.
Instrumentation	System.nanoTime around task; frame timing; CPU/memory profiler.	Same timing plus thread count, context switches, allocation and UI frame metrics.
Architecture	View/UI → ViewModel/controller → synchronous computation.	View/UI → ViewModel/controller → executor/Coroutine/WorkManager → result posted to UI.

2.2 Counter-example: more threads is not always faster

Use a tiny workload such as summing 100 integers repeatedly. Solution A performs it directly. Solution B creates a new thread for each tiny task and synchronises results. Thread creation, scheduling, synchronisation and context switching can exceed the computation cost, making B slower and using more energy. The first degrading decision is creating threads for work too small to amortise their overhead. Another valid stress case is multiple threads contending for one lock.

2.3 Measurement plan and output table

Metric	**How to measure**	**A observed value**	**B observed value**
Task duration	Monotonic clock around identical workload.	Fill from run	Fill from run
CPU time / thread count	Android Studio CPU profiler or `top -H`.	Fill from profiler	Fill from profiler
Memory/PSS	`adb shell dumpsys meminfo <package>` before/after.	Fill from run	Fill from run
Frame drops/latency	Android Studio System	Fill from trace	Fill from trace

	Trace/Perfetto; UI frame timeline.		
Predictability	Repeat trials; report median and worst-case time.	Fill from repeated runs	Fill from repeated runs

2.4 Full working day requirement

Prioritise energy efficiency over peak speed: use a bounded executor, reuse threads rather than creating one per task, batch records, defer non-urgent work with WorkManager and avoid repeated polling. Where the OS/hardware permits, background work can be scheduled when constraints are met. Move work to efficiency cores only if the device scheduler/profile exposes this decision; ordinary apps generally should not assume they can directly pin a core. The final choice must follow measured CPU time, wakeups, memory and responsiveness rather than elapsed time alone.

TASK 3 – Runtime behaviour under changing conditions (10 Marks)

Stage A – ART benchmark design

Workload	**Purpose**	**Measure**
CPU-bound loop/algorithm	Exposes JIT/AOT effect on repeated computation.	Cold/warm time and CPU profile.
Allocation-heavy routine	Exposes allocation rate, GC frequency and pause duration.	Heap graph, GC log, allocations and pause durations.
UI list rendering	Exposes UI-thread frame cost and repeated object creation.	Frame timeline, dropped frames and scroll responsiveness.

```
adb shell dumpsys package <your.package.name>
adb shell cmd package compile -m verify -f <your.package.name>
adb shell cmd package compile -m speed-profile -f <your.package.name>
adb logcat -v time | findstr /i "art gc"
adb shell dumpsys meminfo <your.package.name>
```

Compilation command options vary by Android version and shell. Check `adb shell cmd package compile -h` on the target device before use. Run each mode repeatedly under the same temperature, battery, app state and workload; report median and range.

Stage B – Predictions and comparison

Mode	**Prediction**	**Evidence to compare**
verify / interpret-oriented mode	May reduce compile/storage work but make repeated	Compilation state, CPU task time, cold/warm start.

	CPU-bound execution slower; startup may vary.	
speed-profile	Frequently used paths may be compiled based on profile; startup/hot paths may improve.	Profile state, warm workload time, storage.
speed	More code may be AOT compiled; steady-state may improve but compiled storage can increase and compile cost is higher.	Compile output, package size, CPU time.

JIT compiles hot methods during execution; profile-guided AOT compiles selected methods using profile information; Baseline Profiles identify important startup/user paths so they can be compiled earlier. Interpret results cautiously: thermal throttling, cache state and background activity can hide or imitate compilation effects.

Stage C – Production failure

Injected defect	**Detection**	**Runtime/system component**
Unbounded cache or static Activity reference	Heap grows across repeated navigation; LeakCanary or heap dump identifies retained Activity.	ART heap/GC; leak detector analyses object references.
Main-thread blocking call	StrictMode violation, slow frame or ANR trace.	Main-thread watchdog/Activity Manager and system ANR diagnostics.
Severe allocation churn	Frequent GC, long pauses, rising allocation rate or OutOfMemoryError.	ART allocator and garbage collector.

Stage D – Cross-platform decision

Runtime	**Startup/memory**	**UI/native API access**	**Use-case implication**
Android ART Kotlin/Java	Managed heap; GC; startup depends on process/runtime and compilation state.	Android framework and JNI/NDK.	Strong Android integration; platform-specific

			code required for iOS.
Swift/Objective-C ARC	Native compiled code; reference-count ownership; cycles need weak/unowned references.	Direct Apple framework access.	Strong iOS integration; separate implementation from Android.
Flutter/Dart	Dart AOT in release builds; Flutter rendering engine; memory includes engine/runtime.	Plugins/platform channels bridge to native APIs.	Good shared UI/business logic; test startup, binary size, memory and plugin needs.
React Native / WebView hybrid	JavaScript engine and bridge/JSI or browser WebView overhead depends on design.	Native modules/bridge or web APIs.	Can share UI/business logic, but bridge and rendering choices affect latency and memory.

Recommendation: for a genuine one-codebase requirement, evaluate Flutter first for the same UI and business logic on Android/iOS, while retaining platform-specific adapters for notifications, secure storage and background execution. Accept this only if measured startup, memory, UI frame and native API results meet the product limits. Native apps remain preferable if the benchmarks show the shared runtime exceeds memory or responsiveness limits.

TASK 4 – Secure mobile application audit (7 Marks)

4.1 Threat model and controls

Threat (STRIDE)	**Asset/attack**	**Mechanism**	**Residual risk**
Spoofing	Stolen auth token impersonates user.	Keystore/Keychain-backed key, short-lived tokens, refresh-token rotation, MFA/biometric-bound key where appropriate.	Unlocked/compromised device can still use active session.
Tampering	Local database or API payload modified.	Authenticated encryption, TLS, server validation and integrity checks.	Rooted OS or compromised server may bypass local controls.

Repudiation	User denies changing a record.	Server audit log, timestamp, actor ID and signed/hashed event metadata.	Logs require protected retention and clock reconciliation.
Information disclosure	Local patient DB extracted.	App sandbox, encrypted DB, hardware-backed key, device lock/data protection.	Rooted or unlocked device may expose data while app is active.
Denial of service	Repeated requests or oversized payloads exhaust resources.	Rate limiting, input bounds, retry backoff and resource quotas.	Distributed abuse or network outage remains possible.
Elevation of privilege	Exported Activity/deep link exposes privileged operation.	Explicit exported settings, signature permissions, strict intent validation and least privilege.	Misconfiguration or trusted-library flaw can remain.
Network interception	Cleartext API traffic read/modified.	TLS with hostname validation; optional pinning with rotation plan.	Compromised endpoint/device trust store or pin rotation error.

4.2 Asset-level decision table

Asset	**Attack vector**	**Mechanism applied**	**Decision and reason**
Auth token	Read from shared file or logs.	Private storage, Keystore/Keychain-backed protection, redaction.	Partially blocked; an active compromised app session may still use it.
Local DB	Copy app files from device.	Encrypted database/data protection and sandbox.	Partially blocked; protection depends on key access and device state.
API traffic	MITM on public Wi-Fi.	TLS certificate/hostname validation.	Blocked for ordinary network interception; endpoint compromise remains.

Exported component	Malicious app sends crafted intent/deep link.	Minimise exported components; validate caller/input; signature permission where appropriate.	Blocked if no unauthorised entry point remains; test each exported route.

4.3 Predict before execution

Selected control: allow cleartext HTTP in the test application. Prediction: API traffic becomes exposed to passive capture and active modification; the network attack path is a local Wi-Fi attacker between client and server; the audit verdict changes from Blocked to Exposed for API traffic. Run only against a controlled test endpoint, capture traffic in the lab, compare the audit output, then restore HTTPS and repeat.

4.4 Third-party library leaks device identifier

The affected asset is the device identifier and associated user metadata; the affected layer is application/network data protection. Sandbox and permission controls do not automatically prevent a library that already runs inside the app from sending data it can access. Detect using static dependency/code analysis, runtime network hooks and packet capture. Remediation: update/remove the library, block the endpoint, minimise identifier access, require TLS, review SDK privacy behaviour and add network-egress tests. TLS protects transit but does not make an unnecessary identifier collection acceptable.

4.5 Biometric login and enterprise BYOD

Add biometric authentication to the threat model as a local user-verification factor, not as a replacement for server authentication. Bind access to a cryptographic key that requires user authentication; use platform APIs and hardware-backed key attestation where available. Update trust boundaries for enterprise and personal profiles: separate app data, credentials and policies, enforce managed configuration, and support remote revocation/wipe of work data. Audit logic must check profile ownership, key protection, biometric fallback, device integrity and data-flow between profiles. Residual risks include compromised OS, social engineering and data exported by an authorised user.

TASK 5 – Three power engineers, one device (7 Marks)

Three techniques and instrumentation

Technique	**Trigger/state information**	**Expected power effect**	**Side effect/evidence**
DVFS / core selection	CPU load, thermal state, performance demand and available core clusters.	Lower frequency/voltage and efficiency-core scheduling can reduce energy for suitable workloads.	Task may take longer; record CPU frequency/thermal data if available and task duration.

Doze/App Standby	Screen-off, stationary/idle state, app usage bucket and system restrictions.	Fewer background wakeups/network windows; less radio and CPU activity.	Notifications/sync can be delayed; use dumpsys deviceidle/batterystats and timestamps.
App-level batching/wakelock discipline	Queue size, job constraints, network type, sensor batch and wake-lock duration.	Fewer radio wakeups and less CPU wake time.	Freshness may decrease; track wakeup count, wakelock time, radio-active time and queue delay.

Measurement protocol

Keep device/emulator, app build, workload, brightness, connectivity and battery/thermal starting state as similar as possible.

Collect a baseline with no policy change; record run duration, wakeups, wakelock time, estimated energy and responsiveness.

Enable only one technique at a time and repeat multiple trials.

Capture `adb shell dumpsys batterystats`, Battery Historian/Android Studio Energy Profiler or the available documented power model.

Record event timestamps and state transitions; report median and variability, not only final battery percentage.

```
adb shell dumpsys batterystats --reset
# Run the fixed workload for the selected test duration
adb shell dumpsys batterystats > batterystats.txt
adb shell dumpsys deviceidle
```

Conflict stress test

Hold a partial wakelock continuously while the screen is off. Prediction: the app's wakelock can keep the CPU available and reduce the benefit of Doze even though OS-level idle policy is enabled. Another conflict is scheduling exact alarms too frequently, which defeats batching. Record wakelock duration, wakeups and radio-active time before/after. This demonstrates that app behaviour at one layer can undermine a system-level power-saving mechanism.

Industry decision

Environment	**Strategy and reasoning**
A Always-on tracking	Use efficient sensor sampling, batching, brief justified foreground service, bounded wakelocks and adaptive rates; continuous location/radio use still has a real energy cost.

B Media/gaming	Prioritise sustained frame rate and thermal control; DVFS/core selection matters most during heavy CPU/GPU work; reduce work when frame budget is exceeded.
C Managed fleet	Use WorkManager/JobScheduler constraints, standard OS policies, central configuration and per-OS/version testing; avoid device-specific assumptions and monitor fleet battery/network patterns.

TASK 6 – Battery-aware, secure field-tracking prototype (9 Marks)

6.1 Problem model

Element	**Definition**
Input	Battery %, charging, network type, motion, queue size, last upload, temperature and location/sensor readings.
Output	Sampling interval, upload action, encrypted queued records, estimated energy/cycle and sync status.
Objective	Maximise useful tracking records delivered per unit energy while maintaining confidentiality and integrity.
Constraints	Full-shift operation, intermittent connectivity, urgent tracking requirements, storage limits, thermal safety and secure data handling.

Battery percentage alone is insufficient: 20% battery while charging on Wi-Fi differs from 20% while hot, stationary and offline. Network cost, motion, queue age, temperature and urgency affect whether to sample, defer or upload.

6.2 Policy states and architecture

Dimension	**States / retained information**
BatteryState	CRITICAL ≤15%; LOW 16–25%; NORMAL >25%; CHARGING overrides low-battery deferral for suitable work.

NetworkState	OFFLINE; METERED (cellular); UNMETERED (Wi-Fi).
MotionState	MOVING; STATIONARY; UNKNOWN.
Initial/default	Battery and network read from OS; motion UNKNOWN until sensor window is available; default is conservative sampling with encrypted local queue and no upload when offline.
Transitions	Re-evaluate on battery/network/charging change, motion window, queue threshold, temperature warning, timer or urgent event.
Security invariant	Encrypt record before queueing; use authenticated TLS before upload; only mark synced after server acknowledgement.

Layered architecture: UI (main thread) → domain policy engine (pure decision function) → encrypted repository/data layer (background executor) → OS services (location/sensors, connectivity, battery, WorkManager/foreground service). UI observes queue and status; no network or database work runs on the main thread.

Policy decision table

Condition	**Sampling interval**	**Upload action**	**Security/energy rationale**
Charging + Wi-Fi + moving	10 s	Upload batch now	Power available and unmetered link; encrypt before queue and TLS upload.
80% + LTE + moving	15 s	Batch after threshold or interval	Preserve useful tracking; avoid a radio wake per record.
20% + LTE + stationary	120 s	Defer until charging/Wi-Fi unless queue age/urgency limit reached	Reduce sensor and radio use.
≤15% + stationary	300 s	Queue only; urgent exception permitted	Critical saver mode; keep records encrypted.

Any battery + offline	Based on motion/battery policy	No network attempt; queue and retry with backoff	Avoid futile radio wakeups; preserve UUIDs.
Device hot / thermal warning	60–120 s, safety bounded	Defer bulk upload until cooler/network window	Reduce sustained CPU/GPS/radio load; obey OS thermal controls.
Low-power mode + moving	60 s or OS-approved rate	Small batches on allowed window	Accept slower freshness for energy and thermal stability.

6.3 Prototype policy simulation

```
def choose_policy(battery, charging, network, motion, hot=False, urgent=False):
    if charging and network == "WIFI":
        interval, upload = 10, "UPLOAD_BATCH"
    elif urgent:
        interval, upload = 5, "UPLOAD_PRIORITY_IF_CONNECTED"
    elif network == "OFFLINE":
        interval, upload = (120 if battery <= 25 else 30), "QUEUE_ONLY"
    elif hot:
        interval, upload = 120, "DEFER_BULK_UPLOAD"
    elif battery <= 15:
        interval, upload = 300, "QUEUE_ONLY"
    elif battery <= 25 and motion == "STATIONARY":
        interval, upload = 120, "DEFER_TO_WIFI_OR_CHARGING"
    elif network == "LTE":
        interval, upload = (15 if motion == "MOVING" else 60), "BATCH_UPLOAD"
    else:
        interval, upload = 30, "BATCH_UPLOAD"
    return interval, upload

scenarios = [
    (80, True,  "WIFI", "MOVING", False, False),
    (80, False, "LTE",  "MOVING", False, False),
    (20, False, "LTE",  "STATIONARY", False, False),
    (12, False, "LTE",  "STATIONARY", False, False),
    (55, False, "OFFLINE", "MOVING", False, False),
    (45, False, "WIFI", "MOVING", True, False),
    (18, False, "LTE", "MOVING", False, True)
]
for s in scenarios:
    interval, upload = choose_policy(*s)
```

```
print(s, "->", interval, "seconds,", upload,
    "| encrypted_before_queue=True | TLS_required_for_upload=True")
```

The code is a policy simulation, not an Android application and not a physical energy measurement. A prototype should estimate energy per cycle using a declared model, e.g. E = E_sensing + E_CPU + E_radio, and calibrate its terms using profiler/Battery Historian or a documented device power model. Do not invent an mAh value without a stated model or measured input.

6.4 Independent validation

Condition	Hand-calculated expected policy	Prototype check
20% battery, stationary, LTE	120 s sampling; defer upload to Wi-Fi/charging.	Compare returned interval/action.
Offline, moving, 80% battery	30 s sampling; queue only; no upload attempt.	Compare returned interval/action.
Charging, Wi-Fi, moving	10 s sampling; upload batch.	Compare returned interval/action.

Independent hand calculation tests the rules separately from the code. If the prototype matches all three cases and boundary cases (15%, 25%, offline, urgent, hot), confidence increases because the decision logic has been checked against a reference rather than only against its own output.

6.5 Scalability and load

Load	Policy evaluations	Wake-ups	Memory/battery observation
100 records	Measure from run; expected roughly one evaluation per triggering event.	Record observed count; batching should reduce repeated network wakeups.	Record peak queue memory and measured/modelled energy.
1,000 records	Measure; verify evaluation count scales with events, not nested repeated scans.	Compare wakeups per 100 records with smaller load.	Check bounded batches, database indexes and memory growth.
10,000 records	Measure under stress; record processing errors and queue backlog.	Ensure retries do not create a wakeup storm.	Check pagination/batching, DB growth, thermal and energy trend.

Batching reduces the fixed cost of repeatedly waking the radio/CPU, establishing a connection and negotiating network activity. It does not make high-frequency tracking free: each additional sensor sample consumes energy and may require GPS/CPU processing; large queues also require storage and later upload. Report evaluations, wakeups, memory, radio time and errors, not elapsed time alone.

6.6 Business requirement change: 5,000 devices and emergency updates

The current battery-saving policy defers uploads and lengthens sampling, so it cannot guarantee updates within a few seconds. The requirement changes from energy efficiency as the dominant objective to bounded delivery latency and reliability during emergencies. Investigate an adaptive priority policy: normal tracking remains batched, while an authenticated emergency event activates a time-limited foreground service where permitted, higher-priority server queue, shorter sampling interval and immediate upload retries with backoff. At fleet scale, use server-side prioritisation and monitoring. Trade-offs include higher energy, heat, cellular cost, location privacy exposure and infrastructure cost; limit emergency mode to authorised events and audit every activation.

6.7 Technical reflection (under 150 words)

No single power-saving feature is optimal because workloads differ in freshness, CPU demand, radio use, motion and urgency. Batching reduces repeated radio wakeups when records can wait, but it is unsuitable for emergency tracking that needs updates within seconds. Doze can save energy during idle periods, yet background work may be delayed. DVFS can lower power under moderate load, but sustained graphics or compute work may need performance and can increase heat. A permanent wakelock can also cancel the benefit of OS idle policies. Therefore, the prototype should choose policy from battery, charging, network, motion, temperature, queue age and urgency, then validate the choice using wakeup, memory, latency and energy evidence. Measurements from the actual device are needed to confirm the trade-off.

FINAL SUBMISSION CHECKLIST

Part I: all six design tasks include architecture, tables, traces, constraints and adversarial reasoning.

Part II: collect real device/emulator outputs, profiler screenshots, command logs and measured values for Tasks 1–5.

Run the Task 6 simulation and include its console output; if required by the instructor, port the policy to an Android prototype.

Replace every “Fill from run” field with your own measurements and label device model, OS version, app build, test duration and conditions.

Keep the original question pages followed by this separate answer section.